

Decentralized Computing and Blockchains

Bitcoin and Nakamoto Consensus

Alysson Bessani

DI-FCUL

Permissioned vs Permissionless Ledgers/Blockchains

Permissioned Ledgers

- To participate, a party needs to have a verifiable identity (ID) and be accepted in the network
 - This ID usually is accompanied with a certified public-private key pair, used to establish authenticated channels
 - Sybil attacks are not a concern with verifiable IDs
- Not required to scale to thousands of nodes
 - E.g., 100 may be enough...
- Employ traditional Byzantine consensus protocols:
 - All nodes know all other nodes (as well as n and f)
 - The adversarial threshold is structural: the protocol (sometimes called *consortium consensus*) works if the number of faulty nodes is less than a third of the total nodes ($n > 3f$)
- This is the blockchain architecture preferred by existing companies as it removes intermediaries and is not related to dark web or illegal activities
 - Sometimes it is called DLT (Distributed Ledger Technology) to differentiate from open blockchains such as Ethereum

Permissionless (Open) Ledgers

- Each party knows only a portion of the network
 - Participations fluctuate (nodes join and leave)
 - The parameters of the system are estimated on the observed activity
- Each party can create as many identities as it want
 - This make the system susceptible to Sybil attacks, where a single malicious party act as a many
 - A key issue in permissionless systems is sybil mitigation, therefore the Proof-of-* paradigm, to make nodes spend some resource to participate
- Communication is based on dynamic P2P networks
- This is the standard blockchain model
- Baseline protocol is Nakamoto Consensus, introduced in Bitcoin

The Bitcoin Network

The system that started it all (2008)

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

<https://bitcoin.org/bitcoin.pdf>

Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

1. Introduction

Commerce on the Internet has come to rely almost exclusively on financial institutions serving as

From Satoshi's paper

- A purely peer-to-peer version of electronic cash would allow online payments to be sent *directly from one party to another without going through a financial institution*.
- *A trusted third party is not required* to prevent double- spending.
- We propose a *solution* to the double-spending problem *using a peer-to-peer network*.
- *The network timestamps transactions* by hashing them into an ongoing chain of hash-based proof-of-work, forming *a record that cannot be changed without redoing the proof-of-work*.

From Satoshi's paper

(cont.)

- The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a *majority of CPU power is controlled by nodes that are not cooperating to attack the network*, they'll generate the longest chain and outpace attackers.
- The network itself requires minimal structure. Messages are broadcast on a best-effort basis, and *nodes can leave and rejoin the network at will*, accepting the longest proof-of-work chain as proof of what happened while they were gone.

The Bitcoin Network

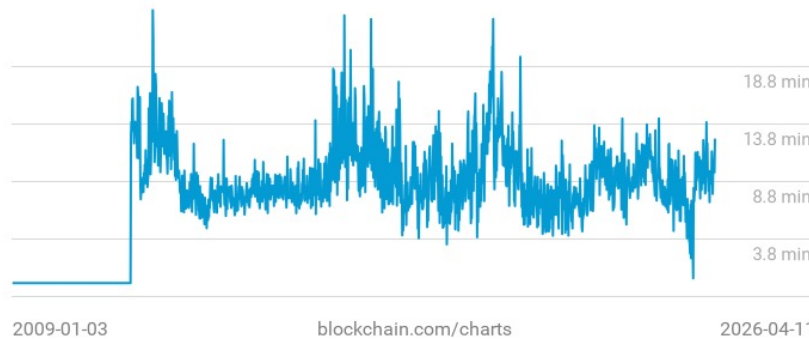
- Bitcoin is mostly a cryptocurrency infrastructure with limited programmability, not a smart contract platform
- The blockchain itself is used to prevent double spending of the BTC cryptocurrency
- Bitcoin users have public and private keys that are associated to “coins” (UTXOs - Unspent Transaction Outputs), generated by the execution of transactions
 - This is different from the account mode of Ethereum and normal banks
- Transactions have input and output UTXOs, with the difference between the total value of inputs and outputs being the **transaction fee**
- The Script language is used to define in which conditions UTXOs can be used, e.g., once a signature is provided (P2PKH), once multiple signatures are provided (Multisig), once a certain time is reached (Time-locks)
- To keep the network producing one block ~ every 10 minutes, nodes programmatically adjust the network block generation difficulty at every 2016 blocks (~ every two weeks)

The Bitcoin Cryptocurrency (\$BTC)

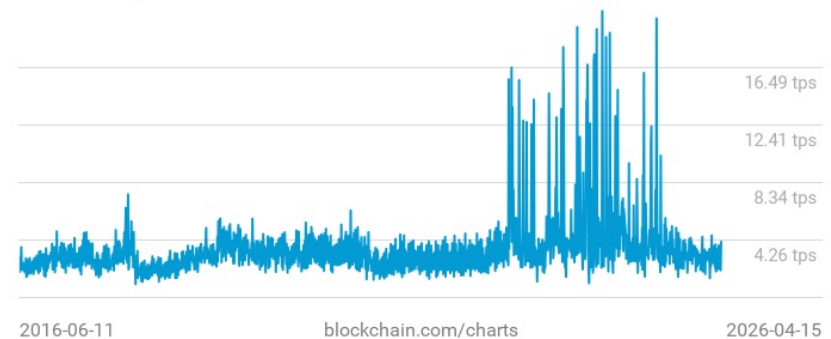
- Bitcoin is designed to handle no more than 21 million BTC
 - The money supply is fixed, i.e., non-inflationary
- Such bitcoins are distributed as block rewards to the block miners
- Block rewards serve as a reward for nodes become miners
 - More honest miners -> more security
 - Successful block producers also receive transaction fees
- Block rewards are halved every 210000 blocks (~ four years)
 - Started with 50BTC, now it is 3.125 BTC, and will be zero in 2140
 - At that point, the reward will exclusively be based on transaction fees

Some numbers about the bitcoin network

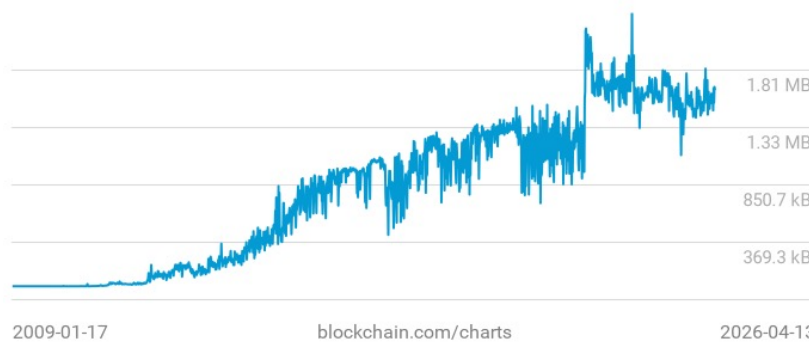
Median Confirmation Time
12.5 min



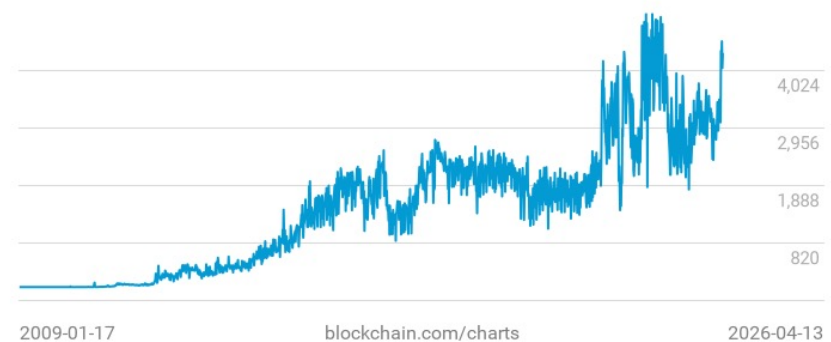
Transaction Rate
4.13 tps



Average Block Size
1.65 MB



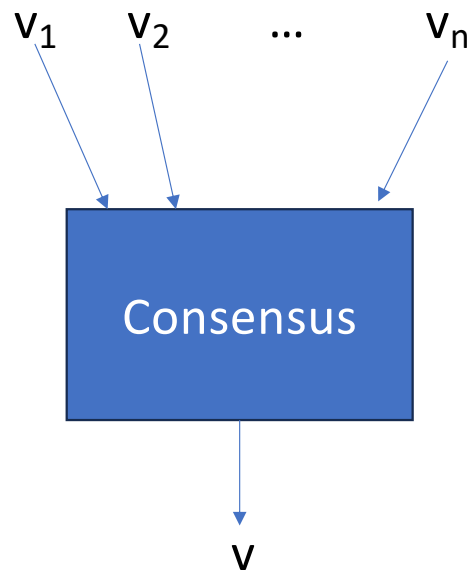
Average Number Of Transactions Per Block
4,331



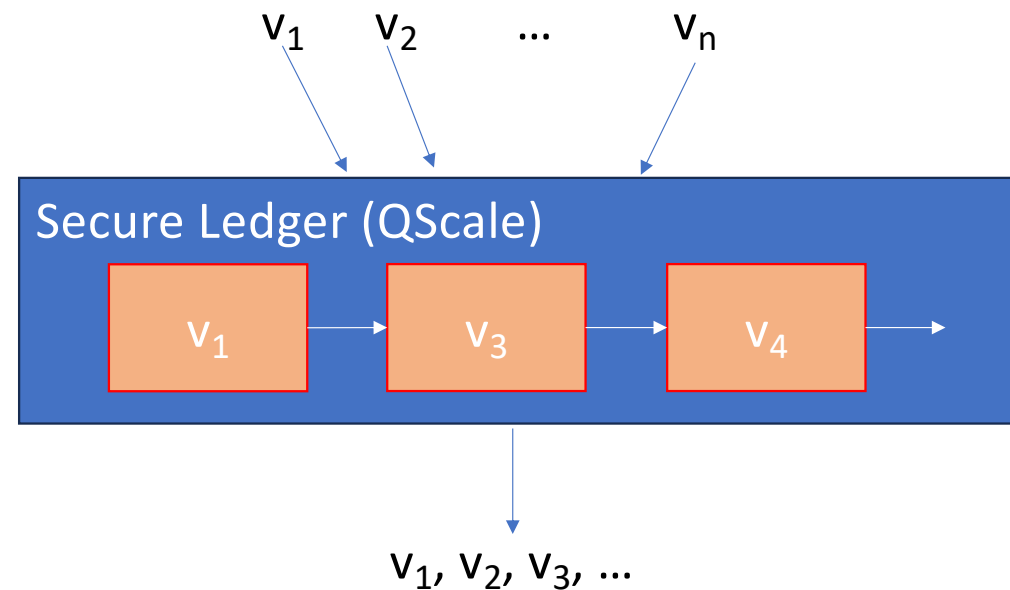
Nakamoto Consensus

Single-shot vs Chained Consensus

- Nakamoto consensus doesn't follow the propose/decide abstraction
- As other blockchains, it is chained consensus protocol
 - There is no clear rule about the commit of transactions/blocks



4/16/26



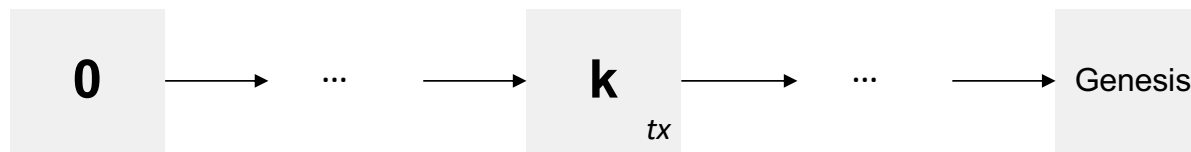
DCB

13

Problem Statement

“A problem well stated is a problem half-solved.” - Charles Kettering

- Consider a system with n parties (honest or not) in which each one of them maintain its copy of the ledger



- A protocol P implements a **robust public transaction ledger** if it satisfies the following two properties:
 - Persistence**: If an honest party reports a ledger that contains a transaction tx in a block more than k positions from the end of the ledger, then tx will be reported in the same position of the ledger by any other honest party
 - Liveness**: If a “valid” transaction tx is sent to all honest parties, then there exist an honest party that will report tx at a block more than k blocks from the end of the ledger

Nakamoto Consensus

- Bitcoin backbone protocol (2008) is the first solution for consensus in open systems
- This protocol is also called Proof-of-Work Consensus (not entirely correct) or Nakamoto Consensus
 - This protocol is being widely studied since 2015
- Key ideas:
 - A block can be added to the blockchain only if a **cryptographic puzzle** is solved
 - New blocks are disseminated in a peer-to-peer network
 - If multiple proposals for extending the chain are received, the longest proposal is used

System Model (used in the formalization)

- Synchronous distributed system with n processes
 - Protocol proceeds in synchronous rounds
 - At most t out of n can be subject to Byzantine failures
 - Static corruptions (adversary picks its t nodes at the beginning)
 - Simplification: In practice, the system is secure if the network latency and the expected time to create a block (blocktime) are bounded
- *Broadcast()/Receive()* implements reliable broadcast
 - A message broadcasted in a round is received in the same round
 - Simplification: In practice, there is a P2P gossip network responsible for delivering these messages within a reasonable time
- Every party can execute at most q queries to a hash function $H(.)$ per round
 - Adversary can execute at most $t \times q$ queries to $H(.)$ on each round
 - Simplification: In practice, each node has limited CPU capacity

The bitcoin backbone protocol (Nakamoto Consensus)

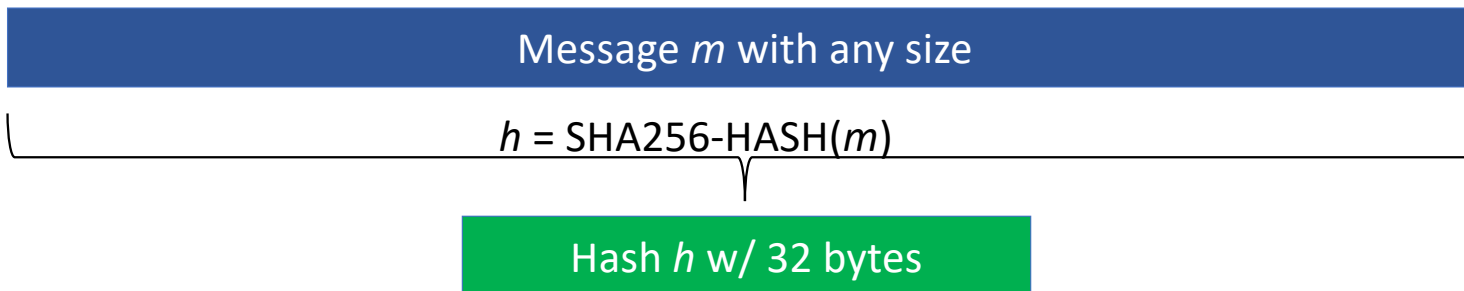
- Local state:
 - C : local copy of the blockchain
- Algorithm:
 - When a new chain C' is received
 - $C = \text{maxvalid}(C, C')$
 - When a new batch of transaction txs is received
 - $C = \text{proof-of-work}(C, txs)$
 - $\text{Broadcast}(C)$
 - When a read request is received
 - Return the transactions on C

Compares two chains and chooses the longest one that is valid, i.e., each block is correctly signed, contains the hash of the previous and solved the proof-of-work puzzle

Solves the following cryptopuzzle: find a valid block containing the transaction and the hash of the previous block such that the hash of the new block is smaller than D (a difficulty parameter)

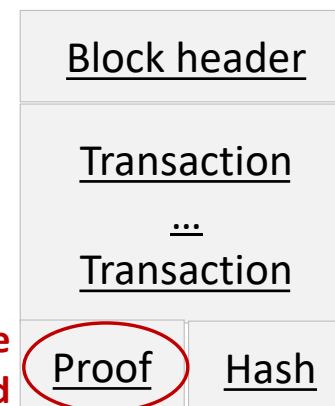
Proof of Work

Cryptographic hash function (e.g., SHA256)



- The Proof-of-Work is generated by changing the block until you find a hash starting with a given number of zeros (difficulty)
- Miners must try a lot, for example, in Bitcoin it takes 10 min. on average for the whole network to generate a block
 - Requires a lot of power!

This field can be changed



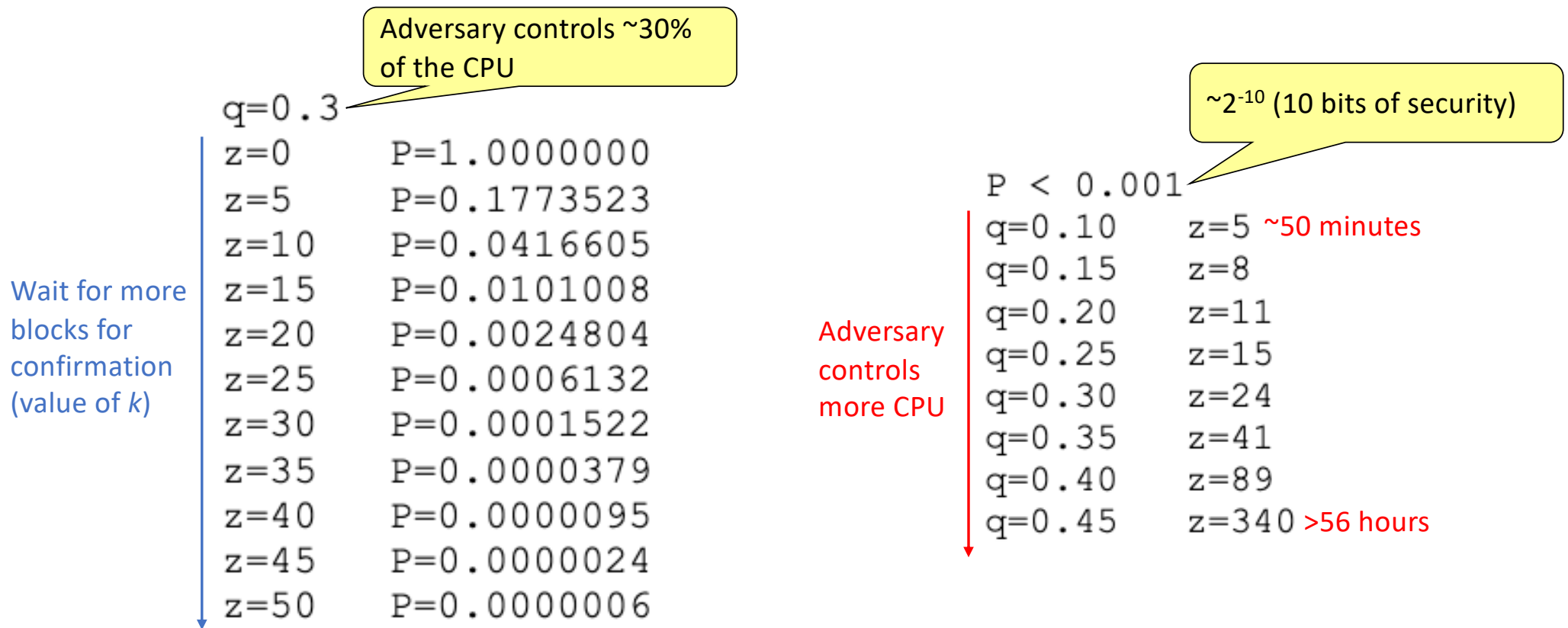
Protocol Properties (*all probabilistic*)

- Consistency (or Common Prefix): at any point, the chains of two honest parties can differ only in the last k blocks;
- Chain Growth: at any point, the chain of honest parties grows by at least k messages in the last k/g rounds;
 - g is called the chain-growth of the protocol;
- Chain quality: for any k consecutive blocks in any chain held by some honest party, the fraction of blocks from honest parties is at least μ (ideally greater than $1/2$).
- Note: both g and μ are dependent on the difficulty of the PoW, which depends on the network latency and the desired blocktime

Bitcoin Security

What is the value of k ?

Nakamoto Consensus Security (as originally stated)

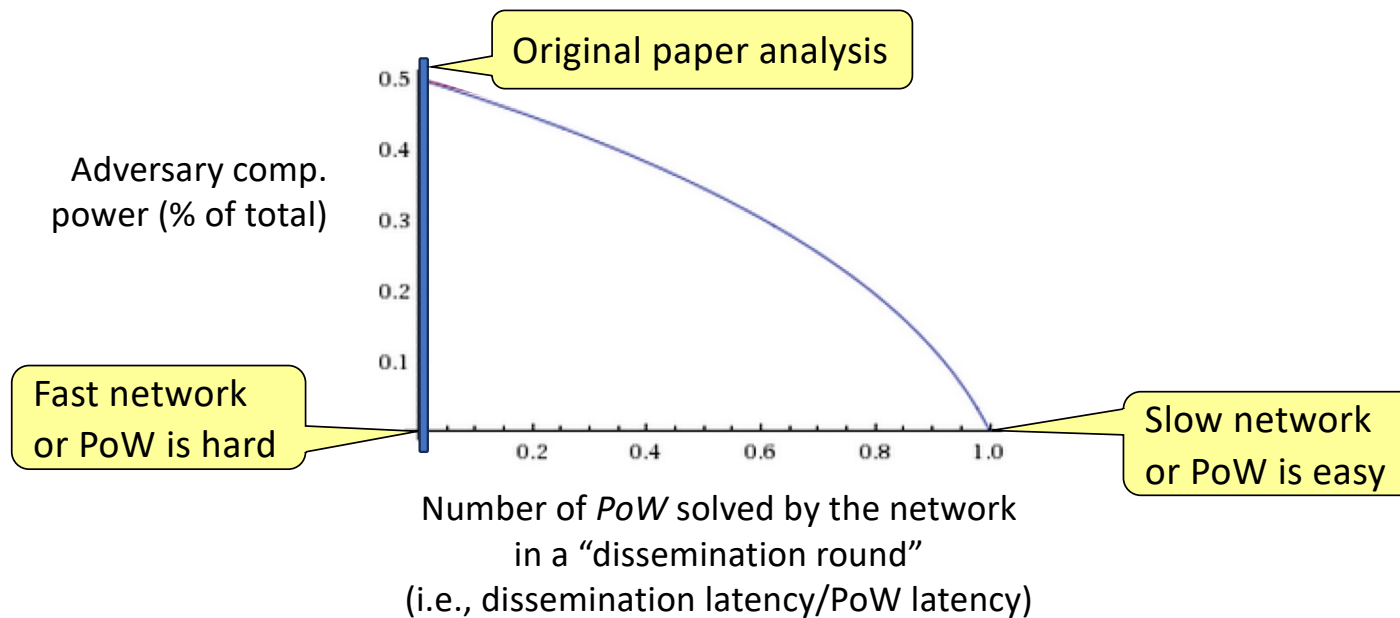


<https://bitcoin.org/bitcoin.pdf> (Section 11)

Actually, it is worse...

NC works **if the adversary controls less than half of the total computing power** and the network disseminates data instantaneously

In practice, the latency of a dissemination round > 0



Summary of Nakamoto Consensus Security

- Is it secure to assume synchronous network on the internet?
 - Yes, if the latency bounds are large and the system self repair itself
- How much corrupted CPU does it tolerate in practice?
 - It is not easy to say, but more than 10% can create problems
 - What is the correct value of k ?
 - $k = 6$ (1 hour) for standard transactions ($<0,35\%$ chance of failure with 10% of CPU under control of the adversary)
 - High-value transactions should wait more, e.g., $k=14$ (2h20m) leads to a failure probability $<0,0002\%$
 - If the adversary controls 25% of CPU, the same probabilities can be obtained with $k=21$ (3h30m) and $k=50$ (8h20m), respectively
- In summary, although the model has its weaknesses, the economical incentives make most Bitcoin nodes follow the protocol

Bibliography

- J. Garay et al. **The Bitcoin Backbone Protocol: Analysis and Applications.** EuroCrypt'15.
- D. Guo, L. Ren. **Bitcoin's Latency-Security Analysis Made Simple.** arXiv 2022.
- S. Nakamoto. **Bitcoin: A Peer-to-Peer Electronic Cash System.** 2009.